**Aluno:** Vinicius Ramon Ferreira **R.A:** 24360978-2

**Atividade - Aula008**

## Exercício em duplas: qual é a saída? (1/3)

Para cada trecho: **(a)** qual é a saída? **(b)** qual conceito da aula explica?

1 · Python

```python
def adicionar(item, lista=[]):
    lista.append(item)
    return lista

print(adicionar(1))
print(adicionar(2))
```

2 · Java

```java
static void zera(int[] v, int n) {
    v[0] = 0;
    n = 0;
}
int[] v = {5, 5}; int n = 5;
zera(v, n);
System.out.println(v[0] + " " + n);
```

Arquivos: exemplos/07_exercicio · execute em onecompiler.com

### 1. Python

(a) Saída:

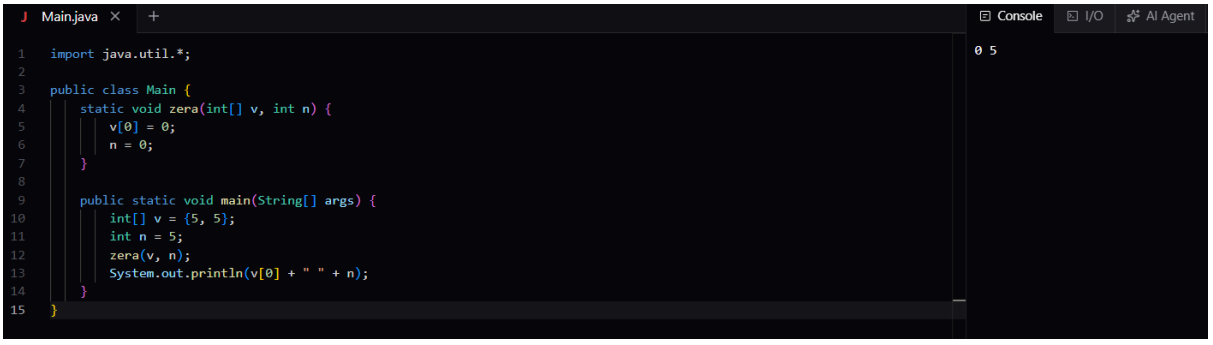



[1]
[1, 2]

(b) Conceito: o valor padrão de um parâmetro é avaliado uma única vez, quando a função é definida (não a cada chamada). Como lista=[] é um objeto mutável, todas as chamadas sem esse argumento compartilham a mesma lista. A correção usual é lista=None e, dentro da função, if lista is None: lista = [].

### 2. Java

(a) Saída:




0 5

(b) Conceito: passagem de parâmetros por valor. Em n, a função recebe uma cópia do inteiro, então n = 0 altera só a cópia local. Em v, a cópia é a do valor da referência, que aponta para o mesmo array. Por isso v[0] = 0 altera o array original, mas atribuir v = outroArray dentro da função não mudaria nada fora dela.

## Exercício: qual é a saída? (2/3)

Para cada trecho: **(a)** qual é a saída? **(b)** qual conceito da aula explica?

**3 · Python**

```python
fs = [lambda: i for i in range(3)]
print([f() for f in fs])
```

**4 · C**

```c
int contador(void) {
    static int n = 0;
    return ++n;
}
// em main:
contador(); contador();
printf("%d\n", contador());
```

Arquivos: exemplos/07_exercicio · execute em onecompiler.com

**3. Python**

(a) Saída:




[2, 2, 2]

Versão Corrigida:

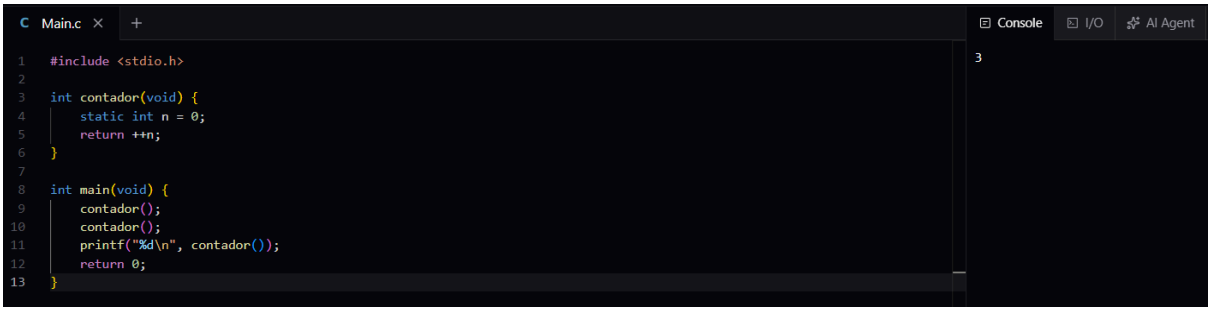



[0, 1, 2]

(b) Conceito: closures com late binding. A lambda não guarda o valor de i, e sim uma referência à variável i do escopo onde foi criada. Ela só é lida quando f() é chamada, e nesse momento o loop já terminou com i = 2. Para capturar o valor de cada iteração, use lambda i=i: i.

**4. C**

(a) Saída:




3

(b) Conceito: variável local estática (static). Ela tem escopo local à função, mas tempo de vida de todo o programa e é inicializada só uma vez. Assim, n mantém o valor entre as chamadas: a 1ª retorna 1, a 2ª retorna 2 e a 3ª (a do printf) retorna 3.

## Exercício: qual é a saída? (3/3)

Para cada trecho: **(a)** qual é a saída? **(b)** qual conceito da aula explica?

**5 · Rust**

```
fn dobra(v: Vec<i32>) -> Vec<i32> {
    v.iter().map(|x| x * 2).collect()
}
let v = vec![1, 2, 3];
let d = dobra(v);
println!("{:?} {:?}", v, d);
```

**6 · Python**

```
total = 0

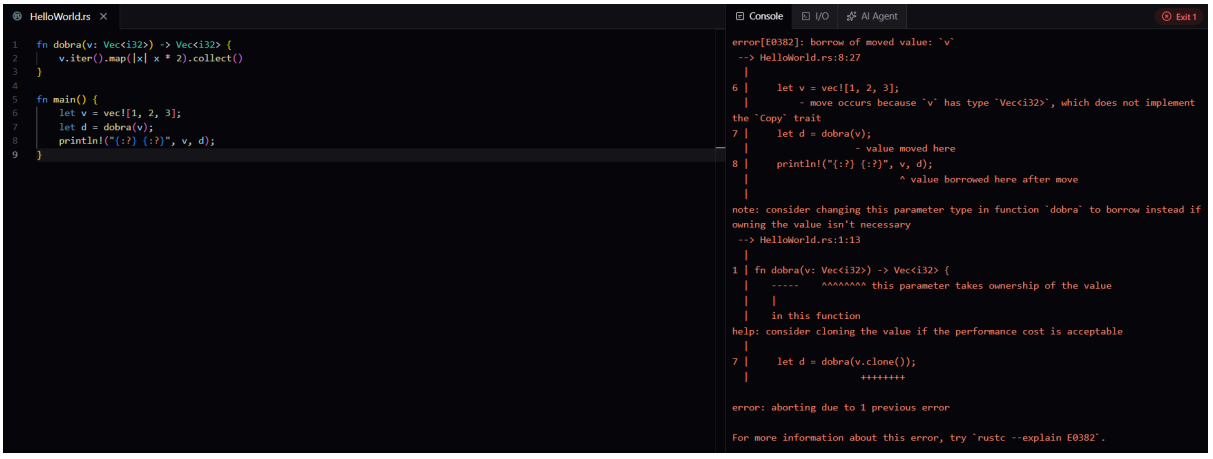
def adiciona(x):
    total = total + x
    return total

print(adiciona(5))
```

Arquivos: exemplos/07_exercicio · execute em onecompiler.com

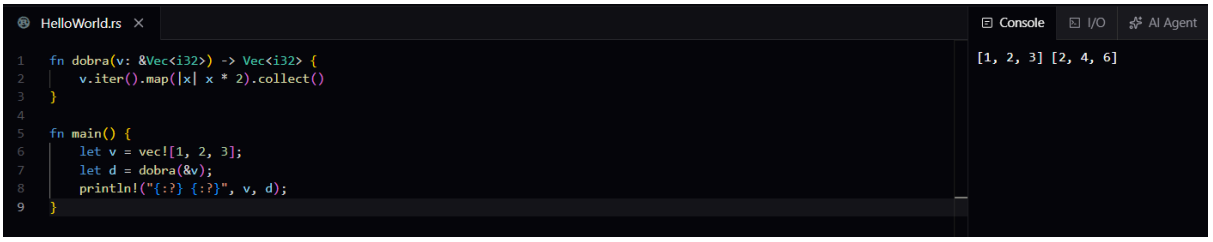
**5. Rust**

(a) Saída:




O código não compila. O erro é borrow of moved value: v (E0382), na linha do println!

Versão Corrigida:




[1, 2, 3] [2, 4, 6]

(b) Conceito: ownership e movimentação (move). dobra(v) recebe Vec<i32> por valor, então o dono do vetor passa a ser a função e v deixa de ser válido em main. Usar v depois do move é erro de compilação. Para funcionar, passe uma referência.

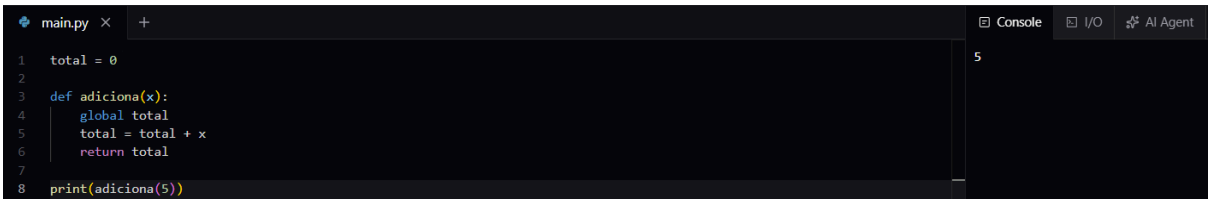
**6. Python**

(a) Saída:




Nada é impresso. Dá erro: UnboundLocalError: cannot access local variable 'total' where it is not associated with a value.

Versão Corrigida:




5

(b) Conceito: regras de escopo. Como existe uma atribuição a total dentro da função, o Python decide, na compilação, que total é local àquela função. Quando total + x é avaliado, a variável local ainda não tem valor, e o total = 0 global é ignorado. Para modificar a global, declare global total dentro da função, e a saída passa a ser 5. Uma alternativa melhor é passar o total como parâmetro e retornar o novo valor.